

Expense Tracker VPS Deployment, Learning, Operations, and Maintenance Manual

Project: Expense Tracker App

Domain: exp.darshanchan.de

Server: Ubuntu VPS

Application Stack: Node.js + Express + SQLite

Process Manager: PM2

Web Server / Reverse Proxy: Nginx

SSL Provider: Let's Encrypt

SSL Automation: Certbot

Backup Solution: Rclone + Google Drive

Backup Scheduler: Cron

Table of Contents

1. Overview
 2. System Architecture
 3. Concepts You Should Understand
 4. What We Built
 5. Initial Deployment Process
 6. Uploading the Application
 7. Understanding node_modules
 8. Installing Dependencies
 9. Running the Application
 10. PM2 Process Management
 11. The EADDRINUSE Error
 12. DNS and Subdomain Setup
 13. Nginx Reverse Proxy Setup
 14. Nginx Troubleshooting
 15. HTTPS and SSL Certificates
 16. Google Drive Backup Setup
 17. Cron-Based Automated Backups
 18. Day-to-Day Operations
 19. Future Update Workflow
 20. Database Update Workflow
 21. Disaster Recovery
 22. Command Reference
 23. Lessons Learned
-

1. Overview

This document records the complete deployment of the Expense Tracker application from a local development machine to a production VPS.

The goal was:

- Host the application online.
- Access it through a subdomain.
- Secure it using HTTPS.
- Keep it running after terminal sessions close.
- Automatically restart after server reboots.
- Automatically back up the database to Google Drive every day.

This document is both:

1. A historical record of what happened.
 2. A learning guide explaining every major concept.
-

2. System Architecture

Current architecture:

Browser ↓ <https://exp.darshanchan.de> ↓ Nginx ↓ localhost:3000 ↓ Node.js Expense Tracker ↓ SQLite Database ↓ Daily Backup Script ↓ Google Drive

Explanation:

- Users visit the website.
 - Nginx receives the request.
 - Nginx forwards it to the Node.js app.
 - The Node.js app reads/writes data from SQLite.
 - Every day a backup is uploaded to Google Drive.
-

3. Concepts You Should Understand

VPS

VPS = Virtual Private Server.

Think of it as a rented computer running in a data center.

Instead of your MacBook running the application, the VPS runs it 24/7.

Ubuntu

Ubuntu is the Linux operating system installed on the VPS.

Comparable to:

- Windows
- macOS

but designed primarily for servers.

SSH

SSH = Secure Shell.

SSH allows you to control the VPS remotely through a terminal.

Example:

```
ssh root@YOUR_SERVER_IP
```

This is equivalent to remotely sitting in front of the server.

Node.js

Node.js allows JavaScript to run outside the browser.

Your Expense Tracker backend runs using Node.js.

Express

Express is a web framework for Node.js.

It handles:

- URLs
- API endpoints

- Web requests
 - Responses
-

SQLite

SQLite is a file-based database.

Instead of a separate database server, all data lives inside:

```
data/expenses.db
```

Advantages:

- Simple
 - Portable
 - Fast
-

Port

Applications listen on ports.

Your app listens on:

```
3000
```

Nginx listens on:

```
80  
443
```

Localhost

localhost means:

"This same machine."

Example:

```
http://localhost:3000
```

means:

"Connect to port 3000 on this VPS."

Reverse Proxy

A reverse proxy sits in front of your application.

Instead of:

```
http://SERVER_IP:3000
```

users access:

```
https://exp.darshanchan.de
```

Nginx forwards requests to the application.

PM2

PM2 is a process manager.

Without PM2:

- Close terminal
- App dies

With PM2:

- App keeps running
 - Auto restart
 - Auto startup after reboot
-

DNS

DNS converts:

exp.darshanchan.de

into:

199.192.16.197

DNS is basically the internet's phonebook.

SSL/TLS

SSL/TLS encrypts traffic.

Without SSL:

http://

With SSL:

https://

Let's Encrypt

Free certificate authority.

Provides SSL certificates.

Certbot

Tool that automatically:

- Requests certificates
 - Installs certificates
 - Renews certificates
-

Rclone

Rclone connects servers to cloud storage.

In our case:

```
VPS → Google Drive
```

Cron

Cron schedules tasks.

Example:

```
Every day at 2 AM
```

Cron runs the backup script automatically.

4. What We Built

Final result:

- Expense Tracker online
 - HTTPS enabled
 - PM2 process management
 - Auto-start on reboot
 - Google Drive backups
 - Daily automated backup schedule
-

5. Initial Deployment Process

Application files were uploaded to:

```
/root/expense-tracker-app
```

Initially the upload included unnecessary files.

Most importantly:

```
node_modules/  
.git/
```

These should generally not be uploaded.

6. Uploading the Application

We initially uploaded too much.

Ideal upload:

```
server.js  
package.json  
package-lock.json  
public/  
data/
```

Avoid uploading:

```
node_modules/  
.git/
```

Reason:

The server can recreate node_modules itself.

7. Understanding node_modules

node_modules contains downloaded packages.

Think of it as:

```
Project source code  
+  
Downloaded libraries
```

The libraries can always be recreated by:

```
npm install
```

Therefore:

Do NOT upload node_modules unless absolutely necessary.

8. Installing Dependencies

Command:

```
npm install
```

Purpose:

Reads:

```
package.json
```

and downloads required packages.

This created:

```
node_modules/
```

on the VPS.

9. Running the Application

Command:

```
node server.js
```

Purpose:

Start the application manually.

Output:

```
Expense tracker running at http://localhost:3000
```

Problem:

Closing terminal would stop the application.

This led us to PM2.

10. PM2 Process Management

Install:

```
npm install -g pm2
```

Start app:

```
pm2 start server.js --name expense-tracker
```

Verify:

```
pm2 list
```

Save startup state:

```
pm2 save
```

Enable startup:

```
pm2 startup
```

Result:

Application survives:

- Terminal closes
- SSH disconnects

- Server reboots
-

11. The EADDRINUSE Error

Error:

```
EADDRINUSE
```

Meaning:

Address already in use.

Cause:

We had:

```
node server.js
```

already running.

Then PM2 tried to start another copy.

Both wanted:

```
port 3000
```

Only one process can own a port.

Diagnosis:

```
lsof -i :3000
```

Output:

```
PID 7432
```

Fix:

```
kill 7432
```

Then:

```
pm2 restart expense-tracker
```

Success.

12. DNS and Subdomain Setup

Created subdomain:

```
exp.darshanchan.de
```

DNS record:

Type:

```
A
```

Host:

```
exp
```

Value:

```
199.192.16.197
```

Result:

Subdomain points to VPS.

13. Nginx Reverse Proxy Setup

Install:

```
apt install nginx
```

Create config:

```
/etc/nginx/sites-available/exp
```

Purpose:

Forward requests from:

```
exp.darshanchan.de
```

to:

```
localhost:3000
```

Enable site:

```
ln -s /etc/nginx/sites-available/exp /etc/nginx/sites-enabled/
```

Test config:

```
nginx -t
```

Reload:

```
systemctl reload nginx
```

14. Nginx Troubleshooting

Error 1

```
server_names_hash_bucket_size
```

Fix:

Uncomment:

```
server_names_hash_bucket_size 64;
```

inside:

```
/etc/nginx/nginx.conf
```

Error 2

Accidentally changed:

```
user www-data;
```

to:

```
ser www-data;
```

Result:

```
unknown directive "ser"
```

Fix:

Restore original line.

15. HTTPS and SSL Certificates

Install:

```
apt install certbot python3-certbot-nginx
```

Request certificate:

```
certbot --nginx -d exp.darshanchan.de
```

Result:

- Certificate created
- Nginx configured automatically
- HTTPS enabled

Site became:

```
https://exp.darshanchan.de
```

16. Google Drive Backup Setup

Install:

```
apt install rclone
```

Configure:

```
rclone config
```

Created remote:

```
gdrive
```

Verified:

```
rclone ls gdrive:
```

17. Cron-Based Automated Backups

Backup script:

```
/root/backup-expenses.sh
```

Purpose:

1. Copy database
2. Create dated filename
3. Upload to Google Drive
4. Remove temporary file

Schedule:

```
0 2 * * * /root/backup-expenses.sh >> /var/log/expense-backup.log 2>&1
```

Meaning:

Every day at:

```
2:00 AM
```

18. Day-to-Day Operations

Check app:

```
pm2 list
```

View logs:

```
pm2 logs expense-tracker
```

Restart app:

```
pm2 restart expense-tracker
```

Stop app:

```
pm2 stop expense-tracker
```

Server reboot:

```
reboot
```

19. Future Update Workflow

Case A: Code Changes Only

Local machine:

Modify code.

Upload changed files.

On VPS:

```
cd /root/expense-tracker-app  
pm2 restart expense-tracker
```

Done.

Case B: New Dependency Added

Example:

```
npm install axios
```

Upload updated:

```
package.json  
package-lock.json
```

On VPS:

```
cd /root/expense-tracker-app  
npm install  
pm2 restart expense-tracker
```

Case C: Frontend Files Changed

Upload:

```
public/
```

Restart:

```
pm2 restart expense-tracker
```

20. Database Update Workflow

Most important section.

Your live database is:

```
/root/expense-tracker-app/data/expenses.db
```

This file contains real user data.

Never overwrite blindly.

Safe Database Change

Before changing:

```
cp expenses.db expenses-backup.db
```

Adding New Columns

Example:

```
ALTER TABLE expenses  
ADD COLUMN category TEXT;
```

This changes structure while keeping data.

Preferred approach.

Replacing Entire Database File

Safe only when:

- No important live data exists
- You intentionally want replacement

Danger:

You may erase data.

Recommended Future Approach

Keep:

Schema Changes

separate from:

Data Changes

Eventually learn database migrations.

21. Disaster Recovery

If VPS dies completely:

1. Create new VPS.
2. Install Node.js.
3. Upload app.
4. Install dependencies.
5. Install PM2.
6. Configure Nginx.
7. Configure HTTPS.
8. Download latest database from Google Drive.
9. Restore:

```
data/expenses.db
```

1. Restart PM2.

System restored.

22. Command Reference

Connect to VPS:

```
ssh root@SERVER_IP
```

Install packages:

```
npm install
```

Run app manually:

```
node server.js
```

Run with PM2:

```
pm2 start server.js --name expense-tracker
```

Restart app:

```
pm2 restart expense-tracker
```

View PM2 apps:

```
pm2 list
```

Check port:

```
lsof -i :3000
```

Kill process:

```
kill PID
```

Test Nginx:

```
nginx -t
```

Reload Nginx:

```
systemctl reload nginx
```

Check backups:

```
rclone ls gdrive:ExpenseTrackerBackups
```

View cron jobs:

```
crontab -l
```

23. Lessons Learned

1. Upload source code, not node_modules.
2. PM2 should own the application process.
3. Always check which process owns a port.

4. Test Nginx before reloading.
 5. Small typos can break configuration files.
 6. HTTPS is easier than it appears when using Certbot.
 7. Backups should be configured before problems happen.
 8. Understanding concepts matters more than memorizing commands.
 9. AI dramatically accelerates learning when combined with real projects.
 10. Building real systems teaches faster than consuming tutorials.
-

Final State Summary

Application URL:

`https://exp.darshanchan.de`

Application Process Manager:

`PM2`

Web Server:

`Nginx`

SSL:

`Let's Encrypt + Certbot`

Database:

`SQLite`

Database File:

`/root/expense-tracker-app/data/expenses.db`

Backups:

Google Drive

Backup Schedule:

Daily at 2:00 AM

Status:

Production-ready personal application deployment completed successfully.